In practice, the value of ϵ is usually taken to be around 0.1.

The solution to this optimization problem produces a linear function of $\mathbf{x}$ accompanied by a band or tube of $\pm\epsilon$ around the function. Points that do not fall inside the tube are the *support vectors*.

11.5.3 Extensions

The optimization problem using quadratic ϵ-insensitive loss can be solved in a similar manner; see Exercise 11.3.

If we formulate this problem using nonlinear transformations of the input vectors, $\mathbf{x} \rightarrow \mathbf{\Phi}(\mathbf{x})$, to a feature space defined by the kernel $K(\mathbf{x}, \mathbf{y})$, then the stationary solution (11.120) is replaced by

$$\boldsymbol{\beta}^* = \sum_{i=1}^{n} (b_i - a_i)\mathbf{\Phi}(\mathbf{x}_i), \tag{11.130}$$

the inner product $\langle \mathbf{x}_i, \mathbf{x}_j \rangle = \mathbf{x}_i^T \mathbf{x}_j$ in (11.120) is replaced by the more general kernel function,

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{\Phi}(\mathbf{x}_i), \mathbf{\Phi}(\mathbf{x}_j) \rangle = \mathbf{\Phi}(\mathbf{x}_i)^T \mathbf{\Phi}(\mathbf{x}_j), \tag{11.131}$$

the matrix $\mathbf{K} = (K(\mathbf{x}_i, \mathbf{x}_j))$ replaces the matrix $\mathbf{K}$ in (11.124), and the SVM regression function (11.122) becomes

$$\mu^*(\mathbf{x}) = \beta_0 + \sum_{i=1}^{n} (b_i - a_i) K(\mathbf{x}, \mathbf{x}_i); \tag{11.132}$$

see Exercise 11.4. Note that $\boldsymbol{\beta}^*$ in (11.130) does not have an explicit representation as it has in (11.120).

11.6 Optimization Algorithms for SVMs

When a data set is small, general-purpose linear programming (LP) or quadratic programming (QP) optimizers work quite well to solve SVM problems; QP optimizers can solve problems having about a thousand points, whereas LP optimizers can deal with hundreds of thousands of points. With large data sets, however, a more sophisticated approach is required.

The main problem when computing SVMs for very large data sets is that storing the entire kernel in main memory dramatically slows down computation. Alternative algorithms, constructed for the specific task of overcoming such computational inefficiencies, are now available in certain SVM software.

We give only brief descriptions of some of these algorithms. The simplest procedure for solving a convex optimization problem is that of gradient ascent:

Gradient Ascent: Start with an initial estimate of the $\boldsymbol{\alpha}$-coefficient vector and then successively update $\boldsymbol{\alpha}$ one α-coefficient at a time using the steepest ascent algorithm.

A problem with this approach is that the solution for $\boldsymbol{\alpha} = (\alpha_1, \cdots, \alpha_n)^\tau$ has to satisfy the linear constraint $\boldsymbol{\alpha}^\tau \mathbf{y} = \sum_{i=1}^n \alpha_i y_i = 0$. Carrying out a non-trivial one-at-a-time update of each α-component (while holding the remaining αs constant at their current values) will violate this constraint, and the solution at each iteration will fall outside the feasible region. The minimum number of αs that can be changed at each iteration is two.

More complicated (but also more efficient) numerical techniques for large learning data sets are now available in many SVM software packages. Examples of such advanced techniques include "chunking," decomposition, and sequential minimal optimization. Each method builds upon certain common elements: (1) choose a subset of the learning set $\mathcal{L}$, (2) monitor closely the KKT optimality conditions to discover which points not in the subset violate the conditions, and (3) apply a suitable optimizing strategy. These strategies are

Chunking: Start with an arbitrary subset (called the "working set" or "chunk") of size 100–500 of the learning set $\mathcal{L}$; use a general LP or QP optimizer to train an SVM on that subset and keep only the support vectors; apply the resulting classifier to all the remaining data in $\mathcal{L}$ and sort the misclassified points by how badly they violate the KKT conditions; add to the support vectors found previously a predetermined number of those points that most violate the KKT conditions; iterate until all points satisfy the KKT conditions. The general optimizer and the point selection process make this algorithm slow and inefficient.

Decomposition: Similar to chunking, except that at each iteration, the size of the subset is always the same; adding new points to the subset means that an equal number of old points must be removed.

Sequential Minimal Optimization (SMO): An extreme version of the decomposition algorithm, whereby the subset consists of only two points at each iteration (see above comments related to the gradient ascent algorithm). These two αs are found at each iteration by using a heuristic argument and then updated so that the constraint $\boldsymbol{\alpha}^\tau \mathbf{y} = \sum_{i=1}^n \alpha_i y_i = 0$ is satisfied and the solution is found within the feasible region.

TABLE 11.4. *Some implementations of SVM.*

Package	Implementation
SVM^{light}	`http://svmlight.joachims.org/`
LIBSVM	`http://csie.ntu.edu.tw/~cjlin/libsvm/`
SVMTorch II	`http://www.idiap.ch/machine-learning.php`
SVMsequel	`http://www.isi.edu/~hdaume/SVMsequel/`
TinySVM	`http://chasen.org/~taku/TinySVM/`

A big advantage of SMO (Platt, 1999) is that the algorithm has an analytical solution and so does not need to refer to a general QP optimizer; it also does not need to store the entire kernel matrix in memory. Although more iterations are needed, SMO is much faster than the other algorithms. The SMO algorithm has been improved in many ways for use with massive data sets.

11.7 Software Packages

There are several software packages for computing SVMs. Many are available for downloading over the Internet. See Table 11.4 for a partial list. Most of these SVM packages use similar data-input formats and command lines.

The most popular SVM package is SVM^{light} by Thorsten Joachims; it is very fast and can carry out classification and regression using a variety of kernels and is used for text classification. It is often used as the basis for other SVM software packages.

The C++–based package LIBSVM by C.-C. Chang and C.-J. Lin, which carries out classification and regression, is based upon SMO and SVM^{light}, and has interfaces to MATLAB, python, perl, ruby, S-Plus (function `svm` in library `libsvm`), and R (function `svm` in library `e1071`); see Venables and Ripley (2002, pp. 344–346). SVMTorch II is an extremely fast C++ program for classification and regression that can handle more than 20,000 observations and more than 100 input variables. SVMsequel is a very fast program that handles classification problems, a variety of kernels (including string kernels), and enormous data sets. TinySVM, which supports C++, perl, ruby, python, and Java interfaces, is based upon SVM^{light}, carries out classification and regression, and can deal with very large data sets.

Bibliographical Notes

There are several excellent references on support vector machines. Our primary references include the books by Vapnik (1998, 2000), Cristianini and Shawe-Taylor (2000), Shawe-Taylor and Cristianini (2004, Chapter 7), Schölkopf and Smola (2002), and Hastie, Tibshirani, and Friedman (2001, Section 4.5 and Chapter 12) and the review articles by Burges (1998), Schölkopf and Smola (2003), and Moguerza and Munoz (2006). An excellent book on convex optimization is Boyd and Vandenberghe (2004).

Most of the theoretical work on kernel functions goes back to about the beginning of the 1900s. The idea of using kernel functions as inner products was introduced into machine learning by Aizerman, Braverman, and Rozoener (1964). Kernels were then put to work in SVM methodology by Boser, Guyon, and Vapnik (1992), who borrowed the "kernel" name from the theory of integral operators.

Our description of string kernels for text categorization is based upon Lodhi, Saunders, Shawe-Taylor, Cristianini, and Watkins (2002). See also Shawe-Taylor and Cristianini (2004, Chapter 11). For applications of SVM to text categorization, see the book by Joachims (2002) and Cristianini and Shawe-Taylor (2000, Section 8.1).

Exercises

11.1 (a) Show that the perpendicular distance of the point (h, k) to the line $f(x, y) = ax + by + c = 0$ is $\pm\ (ah + bk + c)/\sqrt{a^2 + b^2}$, where the sign chosen is that of c.

(b) Let $\mu(\mathbf{x}) = \beta_0 + \mathbf{x}^T\boldsymbol{\beta} = 0$ denote a hyperplane, where $\beta_0 \in \Re$ and $\boldsymbol{\beta} \in \Re^r$, and let $\mathbf{x}_k \in \Re^r$ be a point in the space. By minimizing $\| \mathbf{x} - \mathbf{x}_k \|^2$ subject to $\mu(\mathbf{x}) = 0$, show that the perpendicular distance from the point to the hyperplane is $|\mu(\mathbf{x}_k)|/ \| \boldsymbol{\beta} \|$.

11.2 In the support vector regression problem using a quadratic ϵ-insensitive loss function, formulate and solve the resulting optimization problem.

11.3 The "2-norm soft margin" optimization problem for SVM classification: Consider the regularization problem of minimizing $\frac{1}{2} \| \boldsymbol{\beta} \|^2 + C \sum_{i=1}^n \xi_i^2$ subject to the constraints $y_i(\beta_0 + \mathbf{x}_i^T\boldsymbol{\beta}) \geq 1 - \xi_i$, and $\xi \geq 0$, for $i = 1, 2, \ldots, n$.

(a) Show that the same optimal solution to this problem is reached if we remove the constraints $\xi_i \geq 0$, $i = 1, 2, \ldots, n$, on the slack variables. (Hint: What is the effect on the objective functional if this constraint is violated?)

(*b*) Form the primal Lagrangian F_P, which will be a function of β_0, $\boldsymbol{\beta}$, $\boldsymbol{\xi}$, and the Lagrangian multipliers $\boldsymbol{\alpha}$. Differentiate F_P wrt β_0, $\boldsymbol{\beta}$, and $\boldsymbol{\xi}$, set the results equal to zero, and solve for a stationary solution.

(*c*) Substitute the results from (*b*) into the primal Lagrangian to obtain the dual objective functional F_D. Write out the dual problem (objective functional and constraints) in matrix notation. Maximize the dual wrt $\boldsymbol{\alpha}$. Use the Karush–Kuhn–Tucker complementary conditions $\alpha_i\{y_i(\beta_0+\mathbf{x}_i^\tau\boldsymbol{\beta})-(1-\xi_i)\}=0$ for $i=1,2,\ldots,n$.

(*d*) If $\boldsymbol{\alpha}^*$ is the solution to the dual problem, find $\widehat{\boldsymbol{\beta}}$ and its norm, which gives the width of the margin.

11.4 For the support vector regression problem in a feature space defined by a general kernel function K representing the inner product of pairs of nonlinearly transformed input vectors, formulate and solve the resulting optimization problem using (a) a linear ϵ-insensitive loss function and (b) a quadratic ϵ-insensitive loss function.

11.5 In the support vector regression problem, let $\epsilon = 0$. Consider the quadratic (2-norm) primal optimization problem,

$$\begin{aligned} &\textit{minimize} \quad \lambda \parallel \boldsymbol{\beta} \parallel^2 + \textstyle\sum_{i=1}^n \xi_i^2 \\ &\textit{subject to} \quad y_i - \mathbf{x}_i^\tau\boldsymbol{\beta} = \xi_i, \quad i=1,2,\ldots,n. \end{aligned}$$

Form the Lagrangian, differentiate wrt $\boldsymbol{\beta}$ and ξ_i, $i=1,2,\ldots,n$, and set the results equal to zero for a stationary solution. Substitute these values into the primal functional to get the dual problem. Use $\mathbf{K}$ to represent the Gram matrix with entries either $K_{ij}=\mathbf{x}_i^\tau\mathbf{x}_j$ or $K_{ij}=K(\mathbf{x}_i,\mathbf{x}_j)$. Differentiate the dual functional wrt the Lagrange multipliers $\boldsymbol{\alpha}$, and set the result equal to zero. Show that this solution is related to ridge regression (see Section 5.7.4).

11.6 Let $\mathbf{x},\mathbf{y}\in\Re^2$. Consider the polynomial kernel function, $K(\mathbf{x},\mathbf{y})=\langle\mathbf{x},\mathbf{y}\rangle^2$, so that $r=2$ and $d=2$. Find two different maps $\boldsymbol{\Phi}:\Re^2\to\mathcal{H}$ for $\mathcal{H}=\Re^3$.

11.7 Let $z\in\Re$ and define the $(2m+1)$-dimensional $\boldsymbol{\Phi}$-mapping,

$$\boldsymbol{\Phi}(z)=(2^{-1/2},\cos z,\cdots,\cos mz,\sin z,\cdots,\sin mz)^\tau.$$

Using this mapping, show that the kernel $K(x,y)=\langle\boldsymbol{\Phi}(x),\boldsymbol{\Phi}(y)\rangle$, $x,y\in\Re$, reduces to the *Dirichlet kernel* given by

$$K(x,y)=\frac{\sin((m+\frac{1}{2})\delta)}{2\sin(\delta/2)},$$

where $\delta=x-y$.

11.8 Show that the homogeneous polynomial kernel, $K(\mathbf{x},\mathbf{y})=\langle\mathbf{x},\mathbf{y}\rangle^d$, satisfies Mercer's condition (11.54).

11.9 If K_1 and K_2 are kernels and $c_1, c_2 \geq 0$ are real numbers, show that the following functions are kernels:

(a) $c_1 K_1(\mathbf{x}, \mathbf{y}) + c_2 K_2(\mathbf{x}, \mathbf{y})$;

(b) $K_1(\mathbf{x}, \mathbf{y}) K_2(\mathbf{x}, \mathbf{y})$;

(c) $\exp\{K_1(\mathbf{x}, \mathbf{y})\}$.

(Hint: In each case, you have to show that the function is nonnegative-definite.)

11.10 Prove that in finite-dimensional input space, a symmetric function $K(\mathbf{x}, \mathbf{y})$ is a kernel function iff $\mathbf{K} = (K(\mathbf{x}_i, \mathbf{x}_j))$ is a nonnegative-definite matrix with nonnegative eigenvalues. (Hint: Use the symmetry and the spectral theorem for $\mathbf{K}$ to show that K is a kernel. Then, show that for a negative eigenvalue, the squared-norm of any point $\mathbf{z} \in \mathcal{H}$ is negative, which is impossible.)

11.11 Show that the functional $F_D(\boldsymbol{\alpha})$ in (11.40) is convex; i.e., show that, for $\theta \in (0, 1)$ and $\boldsymbol{\alpha}, \boldsymbol{\beta} \in \Re^n$,

$$F_D(\theta\boldsymbol{\alpha} + (1-\theta)\boldsymbol{\beta}) \leq \theta F_D(\boldsymbol{\alpha}) + (1-\theta) F_D(\boldsymbol{\beta}).$$

11.12 Apply nonlinear-SVM to a binary classification data set of your choice. Make up a two-way table of values of (C, γ) and for each cell in that table compute the CV/10 misclassification rate. Find the pair (C, γ) with the smallest CV/10 misclassification rate. Compare this rate with results obtained using LDA and that using a classification tree.

12

Cluster Analysis

12.1 Introduction

Cluster analysis, which is the most well-known example of *unsupervised learning*, is a very popular tool for analyzing unstructured multivariate data. Within the data-mining community, cluster analysis is also known as *data segmentation*, and within the machine-learning community, it is also known as *class discovery*. The methodology consists of various algorithms each of which seeks to organize a given data set into homogeneous subgroups, or "clusters." There is no guarantee that more than one such group can be found; however, in any practical application, the underlying hypothesis is that the data form a heterogeneous set that should separate into natural groups familiar to the domain experts.

Clustering is a statistical tool for those who need to arrange large quantities of multivariate data into natural groups. For example, marketers use demographics and consumer profiles in an attempt to segment the marketplace into small, homogeneous groups so that promotional campaigns may be carried out more efficiently; biologists divide organisms into hierarchical orders in order to describe the notion of biological diversity; financial managers categorize corporations into different types based upon relevant financial characteristics; archaeologists group artifacts (e.g., broaches) found in

A.J. Izenman, *Modern Multivariate Statistical Techniques*,
doi: 10.1007/978-0-387-78189-1_12,